

3

Bayesian Decision Theory

We discuss probability theory as the framework for making decisions under uncertainty. In classification, Bayes' rule is used to calculate the probabilities of the classes. We generalize to discuss how we can make rational decisions among multiple actions to minimize expected risk. We also discuss learning association rules from data.

3.1 Introduction

PROGRAMMING COMPUTERS to make inference from data is a cross between statistics and computer science, where statisticians provide the mathematical framework of making inference from data and computer scientists work on the efficient implementation of the inference methods.

Data comes from a process that is not completely known. This lack of knowledge is indicated by modeling the process as a random process. Maybe the process is actually deterministic, but because we do not have access to complete knowledge about it, we model it as random and use probability theory to analyze it. At this point, it may be a good idea to jump to the appendix and review basic probability theory before continuing with this chapter.

Tossing a coin is a random process because we cannot predict at any toss whether the outcome will be heads or tails—that is why we toss coins, or buy lottery tickets, or get insurance. We can only talk about the probability that the outcome of the next toss will be heads or tails. It may be argued that if we have access to extra knowledge such as the exact composition of the coin, its initial position, the force and its direction that is applied to the coin when tossing it, where and how it is caught, and so forth, the exact outcome of the toss can be predicted.

UNOBSERVABLE
VARIABLES
OBSERVABLE VARIABLE

The extra pieces of knowledge that we do not have access to are named the *unobservable variables*. In the coin tossing example, the only *observable variable* is the outcome of the toss. Denoting the unobservables by $\mathbf{z}$ and the observable as x , in reality we have

$$x = f(\mathbf{z})$$

where $f(\cdot)$ is the deterministic function that defines the outcome from the unobservable pieces of knowledge. Because we cannot model the process this way, we define the outcome X as a random variable drawn from a probability distribution $P(X = x)$ that specifies the process.

The outcome of tossing a coin is heads or tails, and we define a random variable that takes one of two values. Let us say $X = 1$ denotes that the outcome of a toss is heads and $X = 0$ denotes tails. Such X are Bernoulli-distributed where the parameter of the distribution p_o is the probability that the outcome is heads:

$$P(X = 1) = p_o \text{ and } P(X = 0) = 1 - P(X = 1) = 1 - p_o$$

Assume that we are asked to predict the outcome of the next toss. If we know p_o , our prediction will be heads if $p_o > 0.5$ and tails otherwise. This is because if we choose the more probable case, the probability of error, which is 1 minus the probability of our choice, will be minimum. If this is a fair coin with $p_o = 0.5$, we have no better means of prediction than choosing heads all the time or tossing a fair coin ourselves!

SAMPLE If we do not know $P(X)$ and want to estimate this from a given sample, then we are in the realm of statistics. We have a *sample*, $\mathcal{X}$, containing examples drawn from the probability distribution of the observables x^t , denoted as $p(x)$. The aim is to build an approximator to it, $\hat{p}(x)$, using the sample $\mathcal{X}$.

In the coin tossing example, the sample contains the outcomes of the past N tosses. Then using $\mathcal{X}$, we can estimate p_o , which is the parameter that uniquely specifies the distribution. Our estimate of p_o is

$$\hat{p}_o = \frac{\#\{\text{tosses with outcome heads}\}}{\#\{\text{tosses}\}}$$

Numerically using the random variables, x^t is 1 if the outcome of toss t is heads and 0 otherwise. Given the sample {heads, heads, heads, tails, heads, tails, tails, heads, heads}, we have $\mathcal{X} = \{1, 1, 1, 0, 1, 0, 0, 1, 1\}$ and the estimate is

$$\hat{p}_o = \frac{\sum_{t=1}^N x^t}{N} = \frac{6}{9}$$

3.2 Classification

We discussed credit scoring in section 1.2.2, where we saw that in a bank, according to their past transactions, some customers are low-risk in that they paid back their loans and the bank profited from them and other customers are high-risk in that they defaulted. Analyzing this data, we would like to learn the class “high-risk customer” so that in the future, when there is a new application for a loan, we can check whether that person obeys the class description or not and thus accept or reject the application. Using our knowledge of the application, let us say that we decide that there are two pieces of information that are observable. We observe them because we have reason to believe that they give us an idea about the credibility of a customer. Let us say, for example, we observe customer’s yearly income and savings, which we represent by two random variables X_1 and X_2 .

It may again be claimed that if we had access to other pieces of knowledge such as the state of economy in full detail and full knowledge about the customer, his or her intention, moral codes, and so forth, whether someone is a low-risk or high-risk customer could have been deterministically calculated. But these are nonobservables and with what we can observe, the credibility of a customer is denoted by a Bernoulli random variable C conditioned on the observables $\mathbf{X} = [X_1, X_2]^T$ where $C = 1$ indicates a high-risk customer and $C = 0$ indicates a low-risk customer. Thus if we know $P(C|X_1, X_2)$, when a new application arrives with $X_1 = x_1$ and $X_2 = x_2$, we can

$$\text{choose } \begin{cases} C = 1 & \text{if } P(C = 1|x_1, x_2) > 0.5 \\ C = 0 & \text{otherwise} \end{cases}$$

or equivalently

$$(3.1) \quad \text{choose } \begin{cases} C = 1 & \text{if } P(C = 1|x_1, x_2) > P(C = 0|x_1, x_2) \\ C = 0 & \text{otherwise} \end{cases}$$

The probability of error is $1 - \max(P(C = 1|x_1, x_2), P(C = 0|x_1, x_2))$. This example is similar to the coin tossing example except that here, the Bernoulli random variable C is conditioned on two other observable variables. Let us denote by $\mathbf{x}$ the vector of observed variables, $\mathbf{x} = [x_1, x_2]^T$. The problem then is to be able to calculate $P(C|\mathbf{x})$. Using *Bayes’ rule*, it can be written as

BAYES’ RULE

$$(3.2) \quad P(C|\mathbf{x}) = \frac{P(C)p(\mathbf{x}|C)}{p(\mathbf{x})}$$

PRIOR PROBABILITY $P(C = 1)$ is called the *prior probability* that C takes the value 1, which in our example corresponds to the probability that a customer is high-risk, regardless of the $\mathbf{x}$ value—It is the proportion of high-risk customers in our customer base. It is called the prior probability because it is the knowledge we have as to the value of C *before* looking at the observables $\mathbf{x}$, satisfying

$$P(C = 0) + P(C = 1) = 1$$

CLASS LIKELIHOOD $p(\mathbf{x}|C)$ is called the *class likelihood* and is the conditional probability that an event belonging to C has the associated observation value $\mathbf{x}$. In our case, $p(x_1, x_2|C = 1)$ is the probability that a high-risk customer has his or her $X_1 = x_1$ and $X_2 = x_2$. It is what the data tells us regarding the class.

EVIDENCE $p(\mathbf{x})$, the *evidence*, is the marginal probability that an observation $\mathbf{x}$ is seen, regardless of whether it is a positive or negative example.

$$(3.3) \quad p(\mathbf{x}) = \sum_C p(\mathbf{x}, C) = p(\mathbf{x}|C = 1)P(C = 1) + p(\mathbf{x}|C = 0)P(C = 0)$$

POSTERIOR PROBABILITY Combining the prior and what the data tells us using Bayes' rule, we calculate the *posterior probability* of the concept, $P(C|\mathbf{x})$, *after* having seen the observation, $\mathbf{x}$.

$$\text{posterior} = \frac{\text{prior} \times \text{likelihood}}{\text{evidence}}$$

Because of normalization by the evidence, the posteriors sum up to 1:

$$P(C = 0|\mathbf{x}) + P(C = 1|\mathbf{x}) = 1$$

Once we have the posteriors, we decide by using equation 3.1. For now, we assume that we know the prior and likelihoods; in later chapters, we discuss how to estimate $P(C)$ and $p(\mathbf{x}|C)$ from a given training sample.

In the general case, we have K mutually exclusive and exhaustive classes; $C_i, i = 1, \dots, K$; for example, in optical digit recognition, the input is a bitmap image and there are ten classes. We have the prior probabilities satisfying

$$(3.4) \quad P(C_i) \geq 0 \text{ and } \sum_{i=1}^K P(C_i) = 1$$

$p(\mathbf{x}|C_i)$ is the probability of seeing $\mathbf{x}$ as the input when it is known to

belong to class C_i . The posterior probability of class C_i can be calculated as

$$(3.5) \quad P(C_i|\mathbf{x}) = \frac{p(\mathbf{x}|C_i)P(C_i)}{p(\mathbf{x})} = \frac{p(\mathbf{x}|C_i)P(C_i)}{\sum_{k=1}^K p(\mathbf{x}|C_k)P(C_k)}$$

BAYES' CLASSIFIER

and for minimum error, the *Bayes' classifier* chooses the class with the highest posterior probability; that is, we

$$(3.6) \quad \text{choose } C_i \text{ if } P(C_i|\mathbf{x}) = \max_k P(C_k|\mathbf{x})$$

3.3 Losses and Risks

It may be the case that decisions are not equally good or costly. A financial institution when making a decision for a loan applicant should take into account the potential gain and loss as well. An accepted low-risk applicant increases profit, while a rejected high-risk applicant decreases loss. The loss for a high-risk applicant erroneously accepted may be different from the potential gain for an erroneously rejected low-risk applicant. The situation is much more critical and far from symmetry in other domains like medical diagnosis or earthquake prediction.

LOSS FUNCTION
EXPECTED RISK

Let us define action α_i as the decision to assign the input to class C_i and λ_{ik} as the *loss* incurred for taking action α_i when the input actually belongs to C_k . Then the *expected risk* for taking action α_i is

$$(3.7) \quad R(\alpha_i|\mathbf{x}) = \sum_{k=1}^K \lambda_{ik}P(C_k|\mathbf{x})$$

and we choose the action with minimum risk:

$$(3.8) \quad \text{Choose } \alpha_i \text{ if } R(\alpha_i|\mathbf{x}) = \min_k R(\alpha_k|\mathbf{x})$$

0/1 LOSS

Let us define K actions $\alpha_i, i = 1, \dots, K$, where α_i is the action of assigning $\mathbf{x}$ to C_i . In the special case of the *0/1 loss* where

$$(3.9) \quad \lambda_{ik} = \begin{cases} 0 & \text{if } i = k \\ 1 & \text{if } i \neq k \end{cases}$$

all correct decisions have no loss and all errors are equally costly. The risk of taking action α_i is

$$R(\alpha_i|\mathbf{x}) = \sum_{k=1}^K \lambda_{ik}P(C_k|\mathbf{x})$$

$$\begin{aligned}
&= \sum_{k \neq i} P(C_k | \mathbf{x}) \\
&= 1 - P(C_i | \mathbf{x})
\end{aligned}$$

because $\sum_k P(C_k | \mathbf{x}) = 1$. Thus to minimize risk, we choose the most probable class. In later chapters, for simplicity, we will always assume this case and choose the class with the highest posterior, but note that this is indeed a special case and rarely do applications have a symmetric, 0/1 loss. In the general case, it is a simple postprocessing to go from posteriors to risks and to take the action to minimize the risk.

In some applications, wrong decisions—namely, misclassifications—may have very high cost, and it is generally required that a more complex—for example, manual—decision is made if the automatic system has low certainty of its decision. For example, if we are using an optical digit recognizer to read postal codes on envelopes, wrongly recognizing the code causes the envelope to be sent to a wrong destination.

REJECT In such a case, we define an additional action of *reject* or *doubt*, α_{K+1} , with $\alpha_i, i = 1, \dots, K$, being the usual actions of deciding on classes $C_i, i = 1, \dots, K$ (Duda, Hart, and Stork 2001).

A possible loss function is

$$(3.10) \quad \lambda_{ik} = \begin{cases} 0 & \text{if } i = k \\ \lambda & \text{if } i = K + 1 \\ 1 & \text{otherwise} \end{cases}$$

where $0 < \lambda < 1$ is the loss incurred for choosing the $(K + 1)$ st action of reject. Then the risk of reject is

$$(3.11) \quad R(\alpha_{K+1} | \mathbf{x}) = \sum_{k=1}^K \lambda P(C_k | \mathbf{x}) = \lambda$$

and the risk of choosing class C_i is

$$(3.12) \quad R(\alpha_i | \mathbf{x}) = \sum_{k \neq i} P(C_k | \mathbf{x}) = 1 - P(C_i | \mathbf{x})$$

The optimal decision rule is to

$$\begin{aligned}
&\text{choose } C_i && \text{if } R(\alpha_i | \mathbf{x}) < R(\alpha_k | \mathbf{x}) \text{ for all } k \neq i \text{ and} \\
&&& R(\alpha_i | \mathbf{x}) < R(\alpha_{K+1} | \mathbf{x}) \\
(3.13) \quad &\text{reject} && \text{if } R(\alpha_{K+1} | \mathbf{x}) < R(\alpha_i | \mathbf{x}), i = 1, \dots, K
\end{aligned}$$

Given the loss function of equation 3.10, this simplifies to

$$(3.14) \quad \begin{array}{ll} \text{choose } C_i & \text{if } P(C_i|\mathbf{x}) > P(C_k|\mathbf{x}) \text{ for all } k \neq i \text{ and} \\ & P(C_i|\mathbf{x}) > 1 - \lambda \\ \text{reject} & \text{otherwise} \end{array}$$

This whole approach is meaningful if $0 < \lambda < 1$: If $\lambda = 0$, we always reject; a reject is as good as a correct classification. If $\lambda \geq 1$, we never reject; a reject is as costly as, or costlier than, an error.

In the case of reject, we are choosing between the automatic decision made by the computer program and human decision that is costlier but assumed to have a higher probability of being correct. Similarly, we can imagine a cascade of multiple automatic decision makers, which as we proceed are costlier but have a higher chance of being correct; we discuss such cascades in chapter 17 where we talk about combining multiple learners.

3.4 Discriminant Functions

DISCRIMINANT
FUNCTIONS

Classification can also be seen as implementing a set of *discriminant functions*, $g_i(\mathbf{x})$, $i = 1, \dots, K$, such that we

$$(3.15) \quad \text{choose } C_i \text{ if } g_i(\mathbf{x}) = \max_k g_k(\mathbf{x})$$

We can represent the Bayes' classifier in this way by setting

$$g_i(\mathbf{x}) = -R(\alpha_i|\mathbf{x})$$

and the maximum discriminant function corresponds to minimum conditional risk. When we use the 0/1 loss function, we have

$$g_i(\mathbf{x}) = P(C_i|\mathbf{x})$$

or ignoring the common normalizing term, $p(\mathbf{x})$, we can write

$$g_i(\mathbf{x}) = p(\mathbf{x}|C_i)P(C_i)$$

DECISION REGIONS

This divides the feature space into K *decision regions* $\mathcal{R}_1, \dots, \mathcal{R}_K$, where $\mathcal{R}_i = \{\mathbf{x} | g_i(\mathbf{x}) = \max_k g_k(\mathbf{x})\}$. The regions are separated by *decision boundaries*, surfaces in feature space where ties occur among the largest discriminant functions (see figure 3.1).

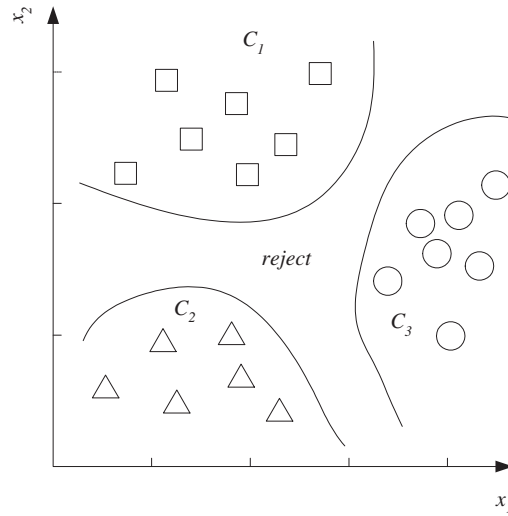


Figure 3.1 Example of decision regions and decision boundaries.

When there are two classes, we can define a single discriminant

$$g(\mathbf{x}) = g_1(\mathbf{x}) - g_2(\mathbf{x})$$

and we

$$\text{choose} \begin{cases} C_1 & \text{if } g(\mathbf{x}) > 0 \\ C_2 & \text{otherwise} \end{cases}$$

An example is a two-class learning problem where the positive examples can be taken as C_1 and the negative examples as C_2 . When $K = 2$, the classification system is a *dichotomizer* and for $K \geq 3$, it is a *polychotomizer*.

DICHOTOMIZER
POLYCHOTOMIZER

3.5 Association Rules

ASSOCIATION RULE

BASKET ANALYSIS

An *association rule* is an implication of the form $X \rightarrow Y$ where X is the *antecedent* and Y is the *consequent* of the rule. One example of association rules is in *basket analysis* where we want to find the dependency between two items X and Y . The typical application is in retail where X and Y are items sold, as we discussed in section 1.2.1.

In learning association rules, there are three measures that are frequently calculated:

SUPPORT ■ *Support* of the association rule $X \rightarrow Y$:

$$(3.16) \quad \text{Support}(X, Y) \equiv P(X, Y) = \frac{\#\{\text{customers who bought } X \text{ and } Y\}}{\#\{\text{customers}\}}$$

CONFIDENCE ■ *Confidence* of the association rule $X \rightarrow Y$:

$$(3.17) \quad \begin{aligned} \text{Confidence}(X \rightarrow Y) \equiv P(Y|X) &= \frac{P(X, Y)}{P(X)} \\ &= \frac{\#\{\text{customers who bought } X \text{ and } Y\}}{\#\{\text{customers who bought } X\}} \end{aligned}$$

LIFT INTEREST ■ *Lift*, also known as *interest* of the association rule $X \rightarrow Y$:

$$(3.18) \quad \text{Lift}(X \rightarrow Y) = \frac{P(X, Y)}{P(X)P(Y)} = \frac{P(Y|X)}{P(Y)}$$

There are other measures as well (Omiecinski 2003), but these three, especially the first two, are the most widely known and used. *Confidence* is the conditional probability, $P(Y|X)$, which is what we normally calculate. To be able to say that the rule holds with enough confidence, this value should be close to 1 and significantly larger than $P(Y)$, the overall probability of people buying Y . We are also interested in maximizing the *support* of the rule, because even if there is a dependency with a strong confidence value, if the number of such customers is small, the rule is worthless. Support shows the statistical significance of the rule, whereas confidence shows the strength of the rule. The minimum support and confidence values are set by the company, and all rules with higher support and confidence are searched for in the database.

If X and Y are independent, then we expect lift to be close to 1; if the ratio differs—if $P(Y|X)$ and $P(Y)$ are different—we expect there to be a dependency between the two items: If the lift is more than 1, we can say that X makes Y more likely, and if the lift is less than 1, having X makes Y less likely.

These formulas can easily be generalized to more than two items. For example, $\{X, Y, Z\}$ is a three-item set, and we may look for a rule, such as $X, Z \rightarrow Y$, that is, $P(Y|X, Z)$. We are interested in finding all such rules having high enough support and confidence and because a sales database

APRIORI ALGORITHM

is generally very large, we want to find them by doing a small number of passes over the database. There is an efficient algorithm, called *Apriori* (Agrawal et al. 1996) that does this, which has two steps: (1) finding frequent itemsets, that is, those which have enough support, and (2) converting them to rules with enough confidence, by splitting the items into two, as items in the antecedent and items in the consequent:

1. To find frequent itemsets quickly (without complete enumeration of all subsets of items), the Apriori algorithm uses the fact that for $\{X, Y, Z\}$ to be frequent (have enough support), all its subsets $\{X, Y\}$, $\{X, Z\}$, and $\{Y, Z\}$ should be frequent as well—adding another item can never increase support. That is, we only need to check for three-item sets all of whose two-item subsets are frequent; or, in other words, if a two-item set is known not to be frequent, all its supersets can be pruned and need not be checked.

We start by finding the frequent one-item sets and at each step, inductively, from frequent k -item sets, we generate candidate $k + 1$ -item sets and then do a pass over the data to check if they have enough support. The Apriori algorithm stores the frequent itemsets in a hash table for easy access. Note that the number of candidate itemsets will decrease very rapidly as k increases. If the largest itemset has n items, we need a total of $n + 1$ passes over the data.

2. Once we find the frequent k -item sets, we need to convert them to rules by splitting the k items into two as antecedent and consequent. Just like we do for generating the itemsets, we start by putting a single consequent and $k - 1$ items in the antecedent. Then, for all possible single consequents, we check if the rule has enough confidence and remove it if it does not.

Note that for the same itemset, there may be multiple rules with different subsets as antecedent and consequent. Then, inductively, we check whether we can move another item from the antecedent to the consequent. Rules with more items in the consequent are more specific and more useful. Here, as in itemset generation, we use the fact that to be able to have rules with two items in the consequent with enough confidence, each of the two rules with single consequent by itself should have enough confidence; that is, we go from one consequent rules to two consequent rules and need not check for all possible two-term consequents (exercise 9).

HIDDEN VARIABLES

Keep in mind that a rule $X \rightarrow Y$ need not imply causality but just an association. In a problem, there may also be *hidden variables* whose values are never known through evidence. The advantage of using hidden variables is that the dependency structure can be more easily defined. For example, in basket analysis when we want to find the dependencies among items sold, let us say we know that there is a dependency among “baby food,” “diapers,” and “milk” in that a customer buying one of these is very much likely to buy the other two. Instead of representing dependencies among these three, we may designate a hidden node, “baby at home,” as the hidden cause of the consumption of these three items. Graphical models that we discuss in chapter 14 allow us to represent such hidden variables. When there are hidden nodes, their values are estimated given the values of observed nodes and filled in.

3.6 Notes

Making decisions under uncertainty has a long history, and over time humanity has looked at all sorts of strange places for evidence to remove the uncertainty: stars, crystal balls, and coffee cups. Reasoning from meaningful evidence using probability theory is only a few hundred years old; see Newman 1988 for the history of probability and statistics and some very early articles by Laplace, Bernoulli, and others who have founded the theory.

Russell and Norvig (2009) give an excellent discussion of utility theory and the value of information, also discussing the assignment of utilities in monetary terms. Shafer and Pearl 1990 is an early collection of articles on reasoning under uncertainty.

Association rules are successfully used in many data mining applications, and we see such rules on many web sites that recommend books, movies, music, and so on. The algorithm is very simple and its efficient implementation on very large databases is critical (Zhang and Zhang 2002; Li 2006). Later, we see in chapter 14 how to generalize from association rules to concepts that need not be binary and where associations can be of different types, also allowing hidden variables.

RECOMMENDATION
SYSTEMS

Recommendation systems are fast becoming one of the major application areas of machine learning. Many retail industries are interested in predicting future customer behavior using past sales data. We can visualize the data as a matrix where rows are the customers, columns

are the items, and entries are the amounts purchased or maybe customer ratings; typically this matrix is very big and also very sparse—most customers have purchased only a very small percentage of the possible items. Though this matrix is very large, it has small rank. This is because there is lot of dependency in the data. People do not shop randomly. People with babies, for example, buy similar things. Certain products are always bought together, or never at the same time. It is these types of regularities, a small number of hidden factors, that makes the matrix low rank. When we talk about dimensionality reduction in chapter 6, we see how we can extract such hidden factors, or dependencies, from data.

3.7 Exercises

1. Assume a disease so rare that it is seen in only one person out of every million. Assume also that we have a test that is effective in that if a person has the disease, there is a 99 percent chance that the test result will be positive; however, the test is not perfect, and there is a one in a thousand chance that the test result will be positive on a healthy person. Assume that a new patient arrives and the test result is positive. What is the probability that the patient has the disease ?

SOLUTION: Let us represent disease by d and test result by t . We are given the following: $P(d = 1) = 10^{-6}$, $P(t = 1|d = 1) = 0.99$, $P(t = 1|d = 0) = 10^{-3}$. We are asked $P(d = 1|t = 1)$.

We use Bayes' rule:

$$\begin{aligned} P(d = 1|t = 1) &= \frac{P(t = 1|d = 1)P(d = 1)}{P(t = 1)} \\ &= \frac{P(t = 1|d = 1)P(d = 1)}{P(t = 1|d = 1)P(d = 1) + P(t = 1|d = 0)P(d = 0)} \\ &= \frac{0.99 \cdot 10^{-6}}{0.99 \cdot 10^{-6} + 10^{-3} \cdot (1 - 10^{-6})} = 0.00098902 \end{aligned}$$

That is, knowing that the test result is positive increased the probability of disease from one in a million to one in a thousand.

LIKELIHOOD RATIO

2. In a two-class problem, the *likelihood ratio* is

$$\frac{p(\mathbf{x}|C_1)}{p(\mathbf{x}|C_2)}$$

Write the discriminant function in terms of the likelihood ratio.

SOLUTION: We can define a discriminant function as

$$g(\mathbf{x}) = \frac{P(C_1|\mathbf{x})}{P(C_2|\mathbf{x})} \text{ and choose } \begin{cases} C_1 & \text{if } g(\mathbf{x}) > 1 \\ C_2 & \text{otherwise} \end{cases}$$

We can write the discriminant as the product of the likelihood ratio and the ratio of priors:

$$g(x) = \frac{p(x|C_1) P(C_1)}{p(x|C_2) P(C_2)}$$

If the priors are equal, the discriminant is the likelihood ratio.

LOG ODDS

3. In a two-class problem, the *log odds* is defined as

$$\log \frac{P(C_1|x)}{P(C_2|x)}$$

Write the discriminant function in terms of the log odds.

SOLUTION: We define a discriminant function as

$$g(x) = \log \frac{P(C_1|x)}{P(C_2|x)} \text{ and choose } \begin{cases} C_1 & \text{if } g(x) > 0 \\ C_2 & \text{otherwise} \end{cases}$$

Log odds is the sum of log likelihood ratio and log of prior ratio:

$$g(x) = \log \frac{p(x|C_1)}{p(x|C_2)} + \log \frac{P(C_1)}{P(C_2)}$$

If the priors are equal, the discriminant is the log likelihood ratio.

4. In a two-class, two-action problem, if the loss function is $\lambda_{11} = \lambda_{22} = 0$, $\lambda_{12} = 10$, and $\lambda_{21} = 5$, write the optimal decision rule. How does the rule change if we add a third action of reject with $\lambda = 1$?

SOLUTION: The loss table is as follows:

Action	Truth	
	C_1	C_2
α_1 : Choose C_1	0	10
α_2 : Choose C_2	5	0

Let us calculate the expected risks of the two actions:

$$R(\alpha_1|x) = 0 \cdot P(C_1|x) + 10 \cdot P(C_2|x) = 10 \cdot (1 - P(C_1|x))$$

$$R(\alpha_2|x) = 5 \cdot P(C_1|x) + 0 \cdot P(C_2|x) = 5 \cdot P(C_1|x)$$

We choose α_1 if

$$R(\alpha_1|x) < R(\alpha_2|x)$$

$$10 \cdot (1 - P(C_1|x)) < 5 \cdot P(C_1|x)$$

$$P(C_1|x) > 2/3$$

If the two misclassifications were equally costly, the decision threshold would be at $1/2$ but because the cost of wrongly choosing C_1 is higher, we want to choose C_1 only when we are really certain; see figure 3.2a and b.

If we add a reject option with a cost of 1, the loss table now becomes

Action	Truth	
	C_1	C_2
α_1 : Choose C_1	0	10
α_2 : Choose C_2	5	0
α_r : Reject	1	1

Let us calculate the expected risks of the three actions:

$$R(\alpha_1|x) = 0 \cdot P(C_1|x) + 10 \cdot P(C_2|x) = 10 \cdot (1 - P(C_1|x))$$

$$R(\alpha_2|x) = 5 \cdot P(C_1|x) + 0 \cdot P(C_2|x) = 5 \cdot P(C_1|x)$$

$$R(\alpha_r|x) = 1$$

We choose α_1 if

$$R(\alpha_1|x) < 1 \Rightarrow P(C_1|x) > 9/10$$

We choose α_2 if

$$R(\alpha_2|x) < 1 \Rightarrow P(C_1|x) < 1/5, \text{ or equivalently if } P(C_1|x) > 4/5$$

We reject otherwise, that is, if $1/5 < P(C_1|x) < 9/10$; see figure 3.2c.

5. Propose a three-level cascade where when one level rejects, the next one is used as in equation 3.10. How can we fix the λ on different levels?
6. Somebody tosses a fair coin and if the result is heads, you get nothing; otherwise, you get \$5. How much would you pay to play this game? What if the win is \$500 instead of \$5?
7. Given the following data of transactions at a shop, calculate the support and confidence values of milk $\rightarrow$ bananas, bananas $\rightarrow$ milk, milk $\rightarrow$ chocolate, and chocolate $\rightarrow$ milk.

Transaction	Items in basket
1	milk, bananas, chocolate
2	milk, chocolate
3	milk, bananas
4	chocolate
5	chocolate
6	milk, chocolate

SOLUTION:

milk $\rightarrow$ bananas : Support = 2/6, Confidence = 2/4

bananas $\rightarrow$ milk : Support = 2/6, Confidence = 2/2

milk $\rightarrow$ chocolate : Support = 3/6, Confidence = 3/4

chocolate $\rightarrow$ milk : Support = 3/6, Confidence = 3/5

Though only half of the people who buy milk buy bananas too, anyone who buys bananas also buys milk.

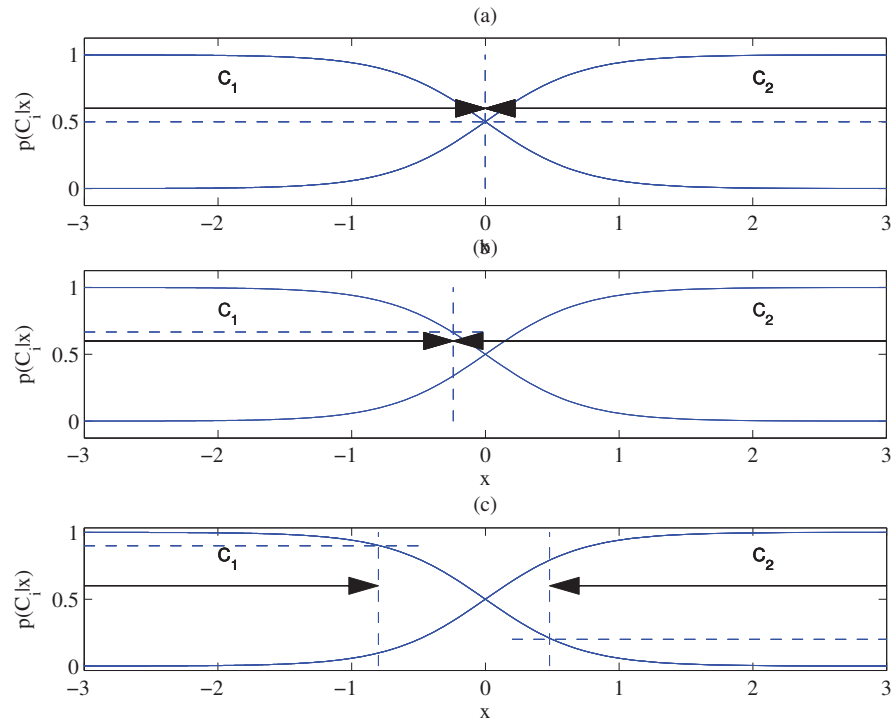


Figure 3.2 The boundary changes as the misclassification losses change. (a) The boundary is where the two posteriors are equal when both misclassifications are equally costly. (b) When the losses are not symmetric, the boundary shifts toward the class that incurs higher risk when misclassified. (c) When there is the option of reject, a region around the boundary is the region of reject.

8. Generalize the confidence and support formulas for basket analysis to calculate k -dependencies, namely, $P(Y|X_1, \dots, X_k)$.
9. Show that as we move an item from the consequent to the antecedent, confidence can never increase: $\text{confidence}(ABC \rightarrow D) \geq \text{confidence}(AB \rightarrow CD)$.
10. Associated with each item sold in basket analysis, if we also have a number indicating how much the customer enjoyed the product, for example, on a scale of 0 to 10, how can you use this extra information to calculate which item to propose to a customer?
11. Show example transaction data where for the rule $X \rightarrow Y$:

- (a) Both support and confidence are high.
- (b) Support is high and confidence is low.
- (c) Support is low and confidence is high.
- (d) Both support and confidence are low.

3.8 References

- Agrawal, R., H. Mannila, R. Srikant, H. Toivonen, and A. Verkamo. 1996. "Fast Discovery of Association Rules." In *Advances in Knowledge Discovery and Data Mining*, ed. U. M. Fayyad, G. Piatetsky-Shapiro, P. Smyth, and R. Uthurusamy, 307–328. Cambridge, MA: MIT Press.
- Duda, R. O., P. E. Hart, and D. G. Stork. 2001. *Pattern Classification*, 2nd ed. New York: Wiley.
- Li, J. 2006. "On Optimal Rule Discovery." *IEEE Transactions on Knowledge and Data Discovery* 18:460–471.
- Newman, J. R., ed. 1988. *The World of Mathematics*. Redmond, WA: Tempus.
- Omiecinski, E. R. 2003. "Alternative Interest Measures for Mining Associations in Databases." *IEEE Transactions on Knowledge and Data Discovery* 15:57–69.
- Russell, S., and P. Norvig. 2009. *Artificial Intelligence: A Modern Approach*, 3rd ed. New York: Prentice Hall.
- Shafer, G., and J. Pearl, eds. 1990. *Readings in Uncertain Reasoning*. San Mateo, CA: Morgan Kaufmann.
- Zhang, C., and S. Zhang. 2002. *Association Rule Mining: Models and Algorithms*. New York: Springer.